Resumo engenharia de software

Esse resumo é feito em três etapas para cada aula, a primeira etapa consiste em um resumo básico, a segunda é um resumo conceitual e a terceira etapa é em nível de prova universitária. Os resumos foram feitos a partir dos três slides que a professora disponibilizou.

📚 ENGENHARIA DE SOFTWARE — AULA 01

1️⃣ RESUMO BÁSICO — PARA QUEM É LEIGO EM TECNOLOGIA

Vamos imaginar que você quer **construir uma casa**.

Você não chegaria em um terreno e falaria:

"Vou começar colocando os tijolos e depois vejo o que acontece."

Primeiro você precisa saber:

- o que o cliente quer;
- como será a casa;
- quais materiais serão necessários;
- como ela será construída;
- testar se tudo está funcionando;
- entregar a casa;
- e depois fazer manutenção.

Com software é a mesma ideia.

Software não é simplesmente "um programa". Segundo o material, software envolve **o programa, a documentação e as configurações necessárias para que ele funcione corretamente**.

💻 Então, o que é Engenharia de Software?

É uma maneira **organizada, planejada e sistemática** de desenvolver, operar e manter softwares. O PDF apresenta a definição do IEEE como a aplicação de uma abordagem **sistemática, disciplinada e quantificável** ao desenvolvimento, operação e manutenção de software.

Em outras palavras:

Engenharia de Software = não sair programando de qualquer jeito.

É utilizar métodos para tentar fazer um software:

- funcionar corretamente;
- atender às necessidades do cliente;
- ter qualidade;
- ser mais fácil de modificar;
- ser desenvolvido dentro do prazo e custo estimados.

🏗️ Como um software é construído?

De acordo com a aula, temos:

Requisitos → Projeto → Implementação → Testes → Implantação → Manutenção.

Vamos traduzir:

📝 1. Requisitos

Descobrir **o que o cliente precisa**.

Exemplo:

"Quero um aplicativo para vender passagens."

Precisamos descobrir:

- quem vai usar;
- o que poderá fazer;
- quais informações serão necessárias;
- quais funcionalidades o sistema deverá possuir.

🏗️ 2. Projeto

Pensar **como o software será construído**.

É como fazer a planta da casa antes de construí-la.

👩‍💻 3. Implementação

É quando o software começa efetivamente a ser **desenvolvido/programado**.

🧪 4. Testes

Verificar se aquilo que foi desenvolvido realmente funciona.

🚀 5. Implantação

Colocar o software para ser utilizado.

🔧 6. Manutenção

Depois que o software está funcionando, o trabalho **não necessariamente termina**.

Podem surgir:

- erros;
- novas necessidades;
- mudanças;
- alterações legais;
- melhorias.

O PDF destaca justamente que "o programa está pronto e em uso" **não significa que o trabalho acabou**. Isso aparece como um dos mitos dos profissionais.

🧠 2️⃣ RESUMO MAIS APROFUNDADO — CONCEITOS TEÓRICOS

Agora vamos sair da explicação "para leigos" e colocar os conceitos que você precisa começar a memorizar para a faculdade.

🔹 SOFTWARE

O material apresenta duas definições importantes.

Sommerville

Software é:

programa + documentação + configuração associada necessária para o funcionamento correto.

Pressman

Software é um conjunto de:

- instruções de computador;
- estruturas de dados;
- documentos.

As instruções produzem os resultados esperados conforme os requisitos definidos.

⚠️ Características importantes

O software:

- **não se desgasta com o tempo**, embora possa se deteriorar;
- pode ser desenvolvido/projetado pela engenharia, diferentemente da manufatura clássica;

- geralmente é feito **sob medida**, em vez de simplesmente ser montado a partir de componentes existentes.

Isso é muito importante porque diferencia software de produtos físicos.

Um celular, por exemplo, pode sofrer desgaste físico.

Um software não "enferruja".

Mas ele pode **se deteriorar** por causa de:

- mudanças;
- novas necessidades;
- alterações no ambiente;
- manutenção inadequada.

🔹 ENGENHARIA DE SOFTWARE

A definição do IEEE é uma das mais importantes para sua prova:

Aplicação de uma abordagem sistemática, disciplinada e quantificável ao desenvolvimento, operação e manutenção de software.

Palavras para decorar:

Sistemática + disciplinada + quantificável

Se aparecer uma questão perguntando o que caracteriza Engenharia de Software, essas três palavras são excelentes pistas.

🔹 POR QUE PRECISAMOS DE ENGENHARIA DE SOFTWARE?

A aula apresenta vários problemas que persistem no desenvolvimento de software:

📈 Demanda

Não conseguimos atender toda a demanda por novos sistemas.

🔧 Manutenção

É difícil manter sistemas existentes diante de novas necessidades e mudanças legais.

💰 Estimativas

Prazo e custo podem ser estimados de maneira imprecisa.

⭐ Qualidade

O produto pode apresentar qualidade abaixo da expectativa.

🗣️ Comunicação

Pode existir dificuldade de comunicação entre cliente e equipe técnica.

⚙️ Produtividade

A produtividade não acompanha a grande demanda por novos sistemas e adaptações.

🔹 "CRISE DO SOFTWARE"

A aula apresenta a ideia de que os problemas históricos do desenvolvimento de software foram associados à chamada **Crise do Software**.

Na década de 1960, houve avanço do hardware e aumento da exigência por softwares mais complexos. O material relaciona esse período ao surgimento da crise.

Em 1968, cientistas da computação ligados à OTAN passaram a utilizar o termo **Engenharia de Software**.

O Pressman, porém, apresenta uma interpretação interessante no material:

não seria exatamente uma "crise", mas uma **"aflição crônica"**.

A ideia é que os problemas persistem durante muito tempo.

🔹 MITOS DO SOFTWARE

Essa parte é **muito importante para prova**.

Mito é uma ideia que parece verdadeira, mas **não corresponde à realidade do desenvolvimento de software**.

O PDF divide os mitos em três grupos:

👔 Mitos do gerenciamento

👤 Mitos dos clientes

👩‍💻 Mitos dos profissionais

👔 MITOS DO GERENCIAMENTO

Mito 1

Ter um manual cheio de padrões e procedimentos significa que a equipe está preparada.

❌ Não necessariamente.

Ter documentação e padrões não significa automaticamente que a equipe sabe desenvolver adequadamente.

Mito 2

Se o projeto atrasar, basta colocar mais programadores.

❌ Não é tão simples.

Adicionar pessoas a um projeto atrasado **não significa automaticamente recuperar o prazo**.

Mito 3

Se terceirizar o software, o gerenciamento pode simplesmente deixar a empresa trabalhar sozinha.

❌ Também é falso.

Terceirizar não elimina a necessidade de acompanhamento.

👤 MITOS DOS CLIENTES

Mito 4

Uma descrição geral dos objetivos é suficiente para começar a programar e os detalhes podem ser definidos depois.

❌ Não.

É necessário compreender adequadamente os requisitos.

Mito 5

Como software é flexível, mudanças de requisitos podem ser facilmente assimiladas.

❌ Mudanças podem gerar impactos no desenvolvimento.

O fato de software ser modificável não significa que qualquer alteração seja simples ou gratuita.

👩‍💻 MITOS DOS PROFISSIONAIS

Mito 6

Terminou o programa e colocou em funcionamento? Acabou o trabalho.

❌ Não.

Existe manutenção.

Mito 7

Não é possível avaliar a qualidade antes do software funcionar.

❌ Falso.

Existem formas de avaliar o software antes da entrada em funcionamento.

Mito 8

O único produto que pode ser entregue é o programa funcionando.

❌ Falso.

O desenvolvimento também produz documentos, projetos e outros resultados.

Mito 9

Engenharia de Software só cria documentação desnecessária e atrasa o trabalho.

❌ Esse é outro mito.

A Engenharia de Software busca organizar o desenvolvimento.

👩‍💻 3️⃣ ENGENHEIRO DE SOFTWARE

O engenheiro de software não é simplesmente "a pessoa que programa".

Segundo a aula, ele utiliza conhecimentos de computadores e computação para **ajudar a resolver problemas**, primeiro procurando entender a natureza do problema e, quando necessário, utilizando tecnologia para implementar a solução.

Isso gera uma ideia muito importante:

O engenheiro de software não deve começar pensando apenas no código. Primeiro precisa entender o problema.

🔎 ANÁLISE E SÍNTESE

A aula apresenta duas ideias:

🔍 Análise

Decompor o problema para entendê-lo.

Imagine:

Problema grande

⬇️

P1 + P2 + P3 + P4

Você divide o problema em partes menores.

🧩 Síntese

Depois de compreender as partes, você busca construir uma **solução**.

O processo pode ser resumido como:

Problema → análise → partes do problema → soluções → solução final.

🛠️ MÉTODOS, FERRAMENTAS, PROCEDIMENTOS E PARADIGMAS

Essa diferença pode cair em prova.

📌 Método

Procedimento formal para produzir um resultado/técnica.

📌 Ferramenta

Instrumento ou sistema automatizado utilizado para realizar uma tarefa.

📌 Procedimento

Combinação de ferramentas e técnicas para produzir determinado resultado.

📌 Paradigma

Uma abordagem ou filosofia específica.

⭐ QUALIDADE DE SOFTWARE

Qualidade não significa simplesmente:

"O programa abre."

A aula relaciona qualidade com dois aspectos:

Qualidade do processo

Qualidade do processo utilizado para criar o produto.

Qualidade do produto

Qualidade do software produzido.

Além disso, a qualidade do produto deve ser considerada dentro do **contexto do ambiente de negócios em que ele será utilizado**.

🖥️ 4️⃣ SISTEMAS

Agora entramos em outro conceito importante.

Segundo o material:

Um sistema é um conjunto de partes ou elementos organizados e relacionados entre si.

Ele recebe **entradas** e produz **saídas**.

Imagine uma máquina de fazer suco:

Frutas + água → processamento → suco

Temos:

Entrada → processamento → saída

📦 TIPOS DE SISTEMAS

A aula apresenta:

- 🔹 Sistemas embutidos (embedded)

São sistemas encapsulados nos dispositivos que controlam.

Exemplos:

- calculadora;
- aplicações de celular;
- estação de jogos;
- termostato.

- 🔹 Sistemas comerciais

Criados para atender necessidades de muitas empresas e/ou pessoas.

Exemplos:

- processadores de texto;
- planilhas eletrônicas;
- sistemas operacionais;
- jogos.

🔹 Sistemas de Informação

São sistemas voltados para coletar, processar, armazenar e distribuir informações para apoiar decisões, coordenação e controle de organizações.

🧩 ELEMENTOS DE UM SISTEMA

Segundo Pfleeger, temos:

Atividades + Objetos + Relações + Fronteira.

🟢 Atividade

Algo que acontece no sistema.

Geralmente é disparada por um gatilho e transforma alguma coisa.

Dica do PDF: normalmente identifique atividade usando um **verbo**.

Exemplos:

- comprar passagem;
- reservar assento;
- fazer check-in.

🔵 Objeto/Entidade

São os elementos envolvidos nas atividades.

Possuem:

- atributos;
- ações.

Exemplo:

Passageiro

- Nome;
- CPF;
- Data de nascimento.

Ação:

- reservar assento.

Outro exemplo:

Voo

- origem;
- destino;
- data;
- hora.

Ação:

- cancelar voo.

🔗 Relação

É aquilo que conecta as atividades aos objetos.

Exemplo:

Passageiro → reservar assento

🚧 Fronteira

Define o **limite do sistema**.

Ou seja, determina o que está dentro e o que está fora daquele sistema.

💾 5️⃣ SISTEMA DE INFORMAÇÃO

Um sistema de informação é um tipo especializado de sistema.

Ele possui componentes inter-relacionados que trabalham para:

Coletar → recuperar → processar → armazenar → distribuir informação

Seu objetivo é auxiliar:

- planejamento;
- controle;
- coordenação;
- análise;
- tomada de decisão.

🔄 Elementos do Sistema de Informação

Essa sequência é **MUITO importante para memorizar**:

1. ENTRADA

Informações que o sistema recebe.

2. PROCESSAMENTO

O sistema manipula/transformationa as entradas.

3. ARMAZENAMENTO

As informações são guardadas.

4. SAÍDA

Resultado produzido.

5. FEEDBACK

Retorno sobre a saída, permitindo avaliar a qualidade do produto.

🧠 Decore assim:

E → P → A → S → F

Entrada → Processamento → Armazenamento → Saída → Feedback

🏠 6️⃣ PROCESSO DE DESENVOLVIMENTO DE SOFTWARE

Essa é provavelmente uma das partes mais importantes da aula.

O professor compara o desenvolvimento de software com a **construção de uma casa**.

Na construção de uma casa, podemos:

1. identificar e analisar requisitos;
2. produzir o projeto;
3. criar especificações;
4. projetar componentes;
5. construir;
6. testar;
7. integrar;
8. realizar modificações;
9. fazer manutenção.

No software:

Requisitos → Projeto → Implementação → Testes → Implantação → Manutenção.

🎓 NÍVEL DE PROVA UNIVERSITÁRIA

Agora imagine que o professor coloque uma questão discursiva:

❓ "Defina Engenharia de Software."

Uma resposta boa seria:

Resposta para prova — Engenharia de Software

Engenharia de Software é a aplicação de uma abordagem sistemática, disciplinada e quantificável ao desenvolvimento, à operação e à manutenção de software. Seu objetivo é organizar o processo de produção de software, buscando desenvolver produtos de qualidade dentro das condições previstas de custo e prazo.

❓ "Explique as etapas do desenvolvimento de software."

Uma resposta completa:

Resposta para prova — Etapas do desenvolvimento

O processo de desenvolvimento de software envolve diferentes etapas. Inicialmente são definidos e analisados os requisitos, buscando compreender as necessidades do sistema. Em seguida, é realizado o projeto, no qual é definida a solução a ser construída. Depois ocorre a implementação, quando o software é desenvolvido. Posteriormente são realizados testes para verificar seu funcionamento. Após a validação, ocorre a implantação do sistema e, finalmente, a manutenção, necessária para corrigir problemas e atender a novas necessidades ou mudanças.

🧠 O QUE EU ACHO MAIS IMPORTANTE DECORAR PARA A PROVA

Se você tiver pouco tempo para estudar, **priorize estes pontos**:

⭐ 1. SOFTWARE

Não é apenas código.

Software = programa + documentação + configuração.

⭐ 2. ENGENHARIA DE SOFTWARE

Abordagem sistemática + disciplinada + quantificável.

⭐ 3. ETAPAS

Requisitos → Projeto → Implementação → Testes → Implantação → Manutenção

⭐ 4. MITOS

Existem:

Gerenciamento + Clientes + Profissionais

E você precisa saber por que cada mito é falso.

⭐ 5. ENGENHEIRO DE SOFTWARE

Entende o problema antes de pensar na solução tecnológica.

⭐ 6. SISTEMA

Partes organizadas e relacionadas → Entrada → processamento → saída

⭐ 7. ELEMENTOS DO SISTEMA

Atividades + Objetos + Relações + Fronteira

⭐ 8. SISTEMA DE INFORMAÇÃO

Entrada → Processamento → Armazenamento → Saída → Feedback

⭐ 9. QUALIDADE

Não é apenas o produto.

Qualidade do processo + qualidade do produto + contexto de utilização.

⭐ 10. PROCESSO DE DESENVOLVIMENTO

O desenvolvimento precisa ser **organizado**, assim como a construção de uma casa: primeiro entender o que será construído, projetar, construir, testar, entregar e manter.

📚 ENGENHARIA DE SOFTWARE — AULA 02

PROCESSOS E MODELOS DE DESENVOLVIMENTO

🟢 PARTE 1 — RESUMO BÁSICO PARA UMA PESSOA LEIGA

🏠 Imagine que você quer construir uma casa

Você tem uma ideia:

"Quero construir uma casa."

Você poderia simplesmente pegar tijolos e começar a colocar um em cima do outro.

Mas provavelmente daria errado. 😅

Você precisaria:

1. descobrir como a casa deve ser;
2. fazer um projeto;
3. decidir quais materiais serão usados;
4. construir;
5. verificar se está tudo certo;
6. entregar;
7. fazer manutenção.

Desenvolver um software funciona de maneira parecida.

Você não deveria simplesmente abrir o computador e começar a escrever código sem saber o que está fazendo.

É por isso que existem os **processos de desenvolvimento de software**.

🔹 O que é um processo?

Segundo o PDF:

Processo é uma sequência de etapas que envolve **atividades, restrições e recursos** para alcançar um resultado desejado.

Traduzindo:

Processo = um jeito organizado de fazer alguma coisa.

Por exemplo, para fazer um bolo:

ingredientes → preparar massa → assar → decorar → bolo

Isso é um processo.

🔹 E por que precisamos de processos?

Porque um processo dá **organização e estrutura** para aquilo que estamos fazendo.

Segundo o material, processos:

- definem as principais atividades;
- utilizam recursos;
- possuem restrições;
- geram produtos intermediários;
- podem possuir subprocessos;
- possuem atividades com entradas e saídas;
- definem a sequência das atividades.

🧠 Em linguagem simples:

Imagine fazer um sanduíche.

Você precisa:

Pegar pão → colocar recheio → fechar → comer

Não faria muito sentido:

Comer → pegar pão → colocar recheio.

O processo organiza **o que fazer e em que ordem fazer**.

🔷 Processo ≠ Procedimento

Essa diferença é MUITO importante.

📌 Processo

É o conjunto maior.

📌 Procedimento

É um **passo a passo de uma etapa**.

🍰 Exemplo do bolo:

PROCESSO: fazer um bolo

Dentro dele temos vários procedimentos:

- selecionar ingredientes;
- escolher utensílios;
- fazer a massa;
- assar;
- etc.

E dentro do procedimento "fazer a massa", temos atividades:

bater os ovos antes de misturar os ingredientes.

Então:

Processo
⬇️
contém **procedimentos**
⬇️
que possuem **atividades**

🔷 Processo × Projeto × Produto

Essa é outra diferença que você precisa guardar.

Imagine uma receita de risoto.

📖 Processo

É o conjunto de procedimentos/forma organizada de fazer.

👨‍🍳 Projeto

É a execução concreta daquele processo.

🍤 Produto

É o resultado final.

O PDF utiliza justamente o exemplo da receita de risoto de camarão, sua execução em determinado dia e pelo chef, e o risoto produzido.

🧠 Pense:

PROCESSO = como fazer

PROJETO = fazer

PRODUTO = aquilo que foi feito

💻 E no desenvolvimento de software?

Existem diferentes maneiras de organizar o desenvolvimento.

Essas maneiras são chamadas de **modelos de processo de software** ou **paradigmas da Engenharia de Software**.

O objetivo é criar **ordem no desenvolvimento**.

🔴 MODELO "CAÓTICO": CODIFICA-REMEDA

Imagine:

"Ah, vamos começar a programar e depois vemos o que acontece."

O desenvolvedor começa a codificar.

Aparece um erro.

Ele conserta.

Aparece outro.

Ele conserta.

O cliente pede alguma coisa.

Ele coloca.

Algo quebra.

Ele remenda.

😂

É o famoso:

codifica → dá problema → remenda → dá outro problema → remenda novamente

O PDF deixa muito claro:

Isso NÃO é um modelo de processo de desenvolvimento de software.

🟠 MODELO CASCATA

Agora imagine uma escada:

Requisitos
⬇️
Projeto
⬇️
Codificação
⬇️
Testes
⬇️
Implantação

⬇️
Manutenção

É chamado de **Cascata** porque as etapas seguem uma sequência.

No modelo apresentado no PDF:

Requisitos → Projeto do Sistema → Projeto do Programa → Codificação → Testes → Aceitação → Operação/Manutenção.

O problema?

Você pode descobrir **muito tarde** que alguma coisa estava errada.

Imagine construir uma casa inteira e só depois perguntar:

"Era para ter dois quartos ou três?"

😢

Por isso o Cascata possui pouca visibilidade para o cliente durante o desenvolvimento.

🟡 MODELO EM V

É uma variação do Cascata.

Ele recebe esse nome porque seu formato lembra a letra:

V

A ideia principal é relacionar as etapas de desenvolvimento com os testes.

A codificação fica no vértice do V.

O modelo busca verificar se:

- o código implementou o projeto;
- o sistema atende ao projeto;
- os requisitos foram implementados.

🧠 Resumindo:

Cascata = sequência

V = sequência + relação explícita entre desenvolvimento e testes

🟢 MODELO DE PROTOTIPAÇÃO

Agora imagine que você quer construir um aplicativo.

Mas o cliente fala:

"Eu não sei exatamente como quero que fique."

Então você cria uma **versão inicial**, um protótipo.

O cliente olha:

"Ah! Gostei disso, mas quero mudar aquilo."

Você altera.

Ele olha novamente:

"Agora sim!"

Esse modelo trabalha com **iterações** para compreender melhor os requisitos.

Seu objetivo é reduzir **risco e incerteza**.

🔵 MODELO ESPIRAL

O modelo Espiral também trabalha com iterações, mas possui uma característica muito importante:

⚠️ GERENCIAMENTO DE RISCOS

O PDF apresenta o modelo de Boehm como uma combinação de:

desenvolvimento + gerenciamento de riscos.

Ou seja:

"Antes de continuar, quais são os riscos desse projeto?"

Ele utiliza protótipos e avaliações de risco antes de avançar.

🟣 MODELO ITERATIVO E INCREMENTAL

Aqui temos duas palavras que parecem complicadas, mas são relativamente simples.

🔄 Iterativo

Significa **repetir/refinar**.

Você faz uma versão.

Analisa.

Melhora.

Faz outra.

Melhora novamente.

➕ Incremental

Significa **entregar partes do sistema aos poucos**.

Imagine um aplicativo:

Versão 1

📅 Calendário

Versão 2

📅 Calendário + 🔔 Lembretes

Versão 3

📅 Calendário + 🔔 Lembretes + 📊 Notas

Versão 4

📅 Calendário + 🔔 Lembretes + 📊 Notas + 🎓 Estágios

Você não espera tudo ficar pronto para entregar alguma coisa.

O PDF destaca justamente que o sistema é projetado e entregue **em partes**, permitindo que o usuário tenha algo disponível mais cedo.

🟪 RUP

RUP significa:

Rational Unified Process

É um método **iterativo e incremental** de desenvolvimento de software.

Durante as iterações, atividades como:

- requisitos;
- projeto;
- implementação;
- testes;

podem ser refinadas.

Cada iteração gera um executável que pode ser entregue ao cliente.

🟦 MÉTODOS ÁGEIS

Aqui chegamos a uma parte MUITO importante da aula.

Os métodos ágeis surgem porque, na prática, os requisitos podem mudar muito.

O cliente pode:

"Eu queria isso."

Depois:

"Pensando bem, quero aquilo."

Depois:

"Usei o sistema e descobri que preciso de outra coisa."

😂

E isso acontece porque os clientes muitas vezes:

- não sabem exatamente o que querem inicialmente;
- têm dificuldade de expressar todas as necessidades;
- descobrem necessidades durante o uso;
- mudam de ideia ao ver o produto sendo construído;
- sofrem influência de mudanças externas e concorrência.

❤️ MANIFESTO ÁGIL

O Manifesto Ágil apresenta quatro valores principais:

👥 Indivíduos e interações

mais que processos e ferramentas

💻 Software funcionando

mais que documentação abrangente

🤝 Colaboração com o cliente

mais que negociação de contratos

🔄 Responder a mudanças

mais que seguir um plano

⚠️ Atenção:

Isso **não significa que processos, ferramentas, documentação, contratos ou planos não importam**.

Significa que, entre os dois lados, o primeiro é considerado **mais importante**.

🟨 SCRUM

O PDF termina chegando ao Scrum.

A definição apresentada pelo Scrum Guide 2020 é:

Scrum é um framework leve que ajuda pessoas, times e organizações a gerar valor por meio de soluções adaptativas para problemas complexos.

Na atividade da aula, os alunos praticamente constroem um Scrum sem perceber.

👫 QUEM FAZ O QUÊ NO SCRUM?

O PDF apresenta três papéis:

👑 Product Owner

Dono do Produto

Decide o que é mais importante.

👨‍💻 Desenvolvedores

Constroem e testam o aplicativo.

🧭 Scrum Master

Auxilia na organização e ajuda a remover impedimentos.

📋 BACKLOG

Imagine que você quer criar um aplicativo e tem 50 ideias.

Você precisa de uma lista.

Essa lista é o:

Backlog do Produto

É o registro das funcionalidades/necessidades do produto.

Depois, selecionamos aquilo que será feito primeiro para o trabalho atual.

Isso aparece no PDF como:

Backlog do Produto → Backlog da Sprint → execução.

⏱️ SPRINT

Sprint é um **período curto de desenvolvimento**.

No material, são apresentados períodos de **1 a 4 semanas**.

Exemplo:

Sprint 1: fazer calendário.

Sprint 2: fazer lembretes.

Sprint 3: fazer consulta de notas.

E assim por diante.

🗣️ REUNIÃO DIÁRIA

Durante o ciclo, a equipe acompanha o trabalho.

O PDF apresenta a **Reunião Diária** como parte desse acompanhamento, associada à ideia de:

Transparência → Inspeção → Adaptação.

🔄 E DEPOIS DA SPRINT?

Quando termina:

📱 Existe uma nova versão funcionando.

Então a equipe:

avalia → recebe feedback → identifica novas necessidades → atualiza o backlog → começa novo ciclo.

É um ciclo contínuo.

🧠 PARTE 2 — RESUMO TEÓRICO MAIS APROFUNDADO

Agora vamos organizar tudo de forma mais "acadêmica".

1. PROCESSO

Processo é uma sequência estruturada de atividades, recursos e restrições destinada a produzir um resultado.

Características:

Elemento	Significado
Atividades	O que precisa ser feito
Recursos	O que será utilizado
Restrições	Limitações/regras
Entradas	O que uma atividade recebe
Saídas	O que uma atividade produz
Sequenciamento	Ordem das atividades
Subprocessos	Processos menores dentro do processo

Essas características são apresentadas diretamente na aula.

2. CICLO DE VIDA DO SOFTWARE

O ciclo de vida é a estrutura que indica os processos e atividades envolvidos no:

desenvolvimento + operação + manutenção

durante toda a vida do sistema.

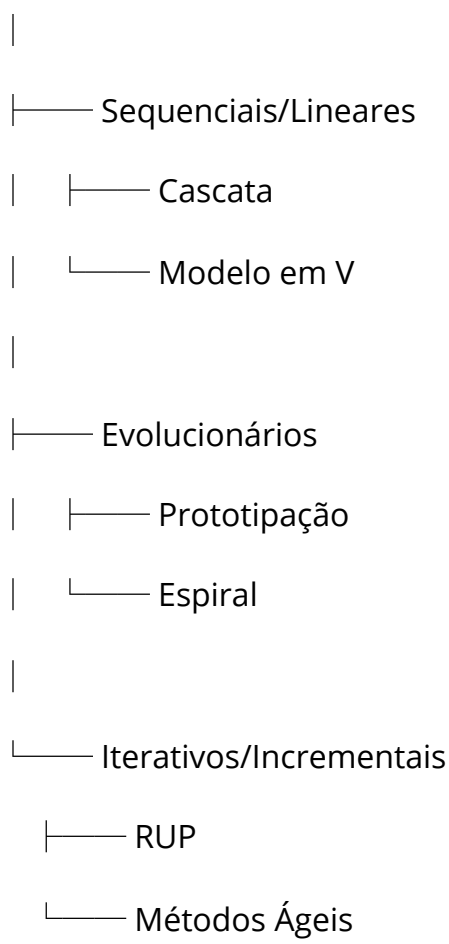
De forma geral:

Requisitos → Projeto → Implementação → Teste → Implantação → Manutenção

3. MODELOS DE PROCESSO

Podemos organizar os modelos da aula assim:

MODELOS DE PROCESSO



Essa classificação aparece no resumo final do PDF.

4. COMPARAÇÃO DOS MODELOS

Modelo	Ideia principal
Cascata	Etapas sequenciais
V	Cascata relacionando desenvolvimento e testes
Prototipação	Iterações para compreender requisitos
Espiral	Iterações + gerenciamento de riscos
Iterativo	Repetir/refinar
Incremental	Entregar partes
RUP	Desenvolvimento iterativo e incremental
Ágil	Valor, colaboração, feedback e adaptação

🎓 PARTE 3 — EXPLICAÇÃO NO NÍVEL DE PROVA UNIVERSITÁRIA

Agora é a parte que eu recomendo você estudar **com bastante atenção**, porque são conceitos que podem aparecer em questões de múltipla escolha ou discursivas.

❓ 1. O que é processo de software?

Resposta de prova:

Resposta de prova — Processo de Software

Um processo é uma sequência de etapas que envolve atividades, restrições e recursos para alcançar um resultado desejado. No desenvolvimento de software, o processo organiza as atividades necessárias para produzir o sistema, estabelecendo entradas, saídas, recursos, restrições e o sequenciamento das atividades.

❓ 2. Diferencie processo, projeto e produto.

Resposta de prova — Processo, Projeto e Produto

Processo corresponde ao conjunto organizado de procedimentos utilizados para alcançar determinado objetivo. Projeto corresponde à concretização ou execução desse processo em uma situação específica. Produto é o resultado obtido a partir da execução do projeto. Assim, pode-se compreender de forma simplificada: processo define como fazer, projeto representa a execução e produto corresponde ao resultado.

❓ 3. Explique o modelo Cascata.

O **Cascata** é um modelo **prescritivo e sequencial**.

As etapas seguem uma ordem e, idealmente, uma etapa começa quando a anterior termina.

Vantagem:

É simples de compreender e explicar, principalmente para clientes pouco familiarizados com desenvolvimento de software.

Desvantagens:

- pouca orientação sobre determinadas situações;
- não representa bem a forma como o código evolui;
- pode exigir muita iteração quando o problema não está bem compreendido;
- baixa visibilidade para o cliente durante o desenvolvimento;
- dificuldade para lidar com mudanças.

🧠 Palavra-chave:

SEQUENCIAL

❓ 4. Qual a diferença entre Cascata e Modelo V?

O Modelo V é uma **variação do Cascata**.

Sua principal característica é relacionar as etapas de desenvolvimento com as etapas de **verificação e validação/testes**.

A codificação ocupa o vértice do V.

Palavra-chave:

Cascata + testes relacionados às etapas de desenvolvimento.

❓ 5. Quando a prototipação é interessante?

Quando existe **incerteza sobre os requisitos**.

O protótipo permite que cliente e desenvolvedor tenham contato com uma representação inicial do produto e realizem iterações para chegar a um consenso sobre o que deve ser desenvolvido.

Palavra-chave:

REQUISITOS INCERTOS

❓ 6. Qual a principal característica do modelo Espiral?

⚠️ GERENCIAMENTO DE RISCOS.

O modelo Espiral, associado a Boehm, combina desenvolvimento e gerenciamento de riscos e utiliza iterações e protótipos para avaliar riscos antes de avançar.

Palavra-chave:

RISCO

❓ 7. Diferencie iterativo de incremental.

Essa é uma questão que eu **apostaria que pode aparecer**.

🔄 Iterativo

O sistema é desenvolvido através de **repetições e refinamentos**.

Você faz:

versão → avaliação → melhoria → nova versão

➕

Incremental

O sistema é desenvolvido e entregue **em partes**.

Exemplo:

Incremento 1 → funcionalidade A

Incremento 2 → funcionalidade B

Incremento 3 → funcionalidade C

O modelo iterativo incremental combina as duas ideias. O PDF destaca desenvolvimento em fases e entrega do sistema em partes.

❓ 8. O que é RUP?

Resposta:

RUP — Rational Unified Process é um método iterativo e incremental de desenvolvimento de software.

As atividades de desenvolvimento podem ser refinadas ao longo de diversas iterações, e cada iteração gera um executável que pode ser entregue ao cliente.

❓ 9. Por que surgiram os métodos ágeis?

Porque o desenvolvimento de software apresenta uma característica importante:

os requisitos podem mudar.

Clientes podem não saber exatamente o que querem inicialmente, podem descobrir novas necessidades durante o uso e podem alterar suas expectativas ao visualizar o produto sendo desenvolvido.

Portanto, métodos que permitem:

feedback + adaptação + entregas frequentes

podem lidar melhor com esse cenário.

❓ 10. Quais são os quatro valores do Manifesto Ágil?

1️⃣ Indivíduos e interações

mais que processos e ferramentas.

2️⃣ Software funcionando

mais que documentação abrangente.

3️⃣ Colaboração com o cliente

mais que negociação de contratos.

4️⃣ Responder a mudanças

mais que seguir um plano.

🧠 Para decorar:

Pessoas → Software → Cliente → Mudança

❓ 11. O que é Scrum?

Segundo o material:

Scrum é um framework leve que ajuda pessoas, times e organizações a gerar valor por meio de soluções adaptativas para problemas complexos.

Na prática da aula, você pode pensar no Scrum como uma forma organizada de trabalhar em **ciclos curtos**, entregando partes do produto, recebendo feedback e adaptando o trabalho.

🧠 RESUMÃO FINAL PARA REVISAR ANTES DA PROVA

Se você estiver estudando correndo, **decore este quadro**:

CONCEITO	LEMBRE
Processo	sequência organizada de atividades
Procedimento	passo a passo de uma etapa
Projeto	execução/concretização do processo
Produto	resultado do projeto
Cascata	sequência linear
Modelo V	Cascata + verificação/validação
Prototipação	entender/refinar requisitos
Espiral	gerenciamento de riscos
Iterativo	repetir e refinar
Incremental	entregar em partes
RUP	iterativo + incremental
Ágil	adaptação + feedback + colaboração
Product Owner	prioriza/decide o que é mais importante
Desenvolvedores	constroem e testam
Scrum Master	ajuda na organização e remoção de impedimentos
Backlog do Produto	lista de funcionalidades/necessidades
Sprint	ciclo curto de trabalho
Reunião Diária	acompanhar andamento
Feedback	permite avaliar e adaptar

🚨 PEGADINHAS QUE VOCÊ PRECISA EVITAR

❌ "Cascata é um modelo que se adapta facilmente às mudanças."

ERRADO.

A principal característica é a sequência das etapas e ele apresenta dificuldades diante de mudanças.

❌ "Prototipação serve principalmente para gerenciamento de riscos."

Cuidado.

Na aula, **Prototipação → compreensão/refinamento dos requisitos e redução de incerteza**.

Espiral → desenvolvimento + gerenciamento de riscos.

❌ "Incremental significa repetir o mesmo processo."

Não.

Iterativo = repetir/refinar.

Incremental = adicionar/entregar partes.

❌ "Métodos ágeis não fazem planejamento."

Errado.

O Manifesto Ágil valoriza **responder a mudanças mais que seguir um plano** — não significa ausência de planejamento.

❌ "Scrum é uma linguagem de programação."

Errado.

É um **framework** para trabalho e desenvolvimento adaptativo.

🌟 A FORMA MAIS FÁCIL DE ENTENDER A AULA INTEIRA

Pense na evolução assim:

😵 **CAOS**
"Vou programar e consertar depois."

⬇️

🌊 **CASCATA**
"Vamos fazer tudo seguindo uma sequência."

⬇️

🔻 **MODELO V**
"Vamos relacionar desenvolvimento e testes."

⬇️

🧪 **PROTOTIPAÇÃO**
"Vamos criar uma versão para descobrir melhor o que o cliente quer."

⬇️

🌀 **ESPIRAL**
"Vamos iterar, mas prestando atenção principalmente aos riscos."

⬇️

🔄 **ITERATIVO + INCREMENTAL**
"Vamos melhorar continuamente e entregar em partes."

⬇️

🚀 **ÁGIL / SCRUM**
"Vamos trabalhar em ciclos curtos, entregar, receber feedback e adaptar."

📚 ENGENHARIA DE SOFTWARE — AULA 03

IDENTIFICAÇÃO DE REQUISITOS E ENGENHARIA DE REQUISITOS

🟢 PARTE 1 — EXPLICAÇÃO BÁSICA PARA UMA PESSOA LEIGA

🏠 Primeiro, imagine que você vai construir uma casa

Imagine que alguém fala para um pedreiro:

"Quero uma casa."

Isso é suficiente?

Não! 😂

O pedreiro precisaria perguntar:

- Quantos quartos?
- Quantos banheiros?
- Terá garagem?
- Qual o tamanho?
- Qual o tipo de piso?
- Terá piscina?
- Qual é o orçamento?
- Em quanto tempo precisa ficar pronta?

Se o pedreiro simplesmente começasse a construir sem perguntar nada, provavelmente faria uma casa **diferente daquela que o cliente imaginava**.

💡 **É exatamente isso que pode acontecer com um software.**

💻 O que são requisitos?

Um **requisito** é, basicamente:

Algo que o sistema precisa fazer ou possuir para atender à necessidade do usuário/cliente.

O PDF apresenta requisito como uma capacidade necessária ao usuário para resolver um problema ou atingir um objetivo, ou uma capacidade necessária para satisfazer contrato, padrão, especificação ou documentação formal.

Exemplo:

O cliente diz:

"Preciso de um sistema para controlar os funcionários."

Isso ainda está muito vago.

O analista precisa descobrir:

- cadastrar funcionário;
- alterar funcionário;
- excluir funcionário;
- consultar funcionário;
- permitir acesso remoto;
- responder rapidamente;
- proteger os dados etc.

Ou seja:

Necessidade → entendimento → requisito

🤔 Por que identificar requisitos é tão importante?

Porque **cliente e desenvolvedor podem entender coisas diferentes**.

O PDF até propõe uma atividade em que todos recebem as mesmas instruções para desenhar algo, mas não podem fazer perguntas nem anotar as instruções. Depois, a turma compara os desenhos.

A ideia da atividade é mostrar:

Mesmo recebendo as mesmas informações, pessoas diferentes podem interpretá-las de maneiras diferentes.

Isso também pode acontecer na construção de um software.

🎯 Um exemplo MUITO bom da aula

O cliente fala:

"Preciso de um programa para calcular a área de um retângulo."

Parece suficiente, certo?

Não!

O PDF mostra que precisamos descobrir coisas como:

- o usuário informa os lados em metros;
- os valores têm duas casas decimais;
- a fórmula é L1 × L2;
- o resultado também tem duas casas decimais;
- existe truncamento;
- o cálculo é acionado pelo usuário.

E ainda podemos precisar saber:

- o que acontece se o usuário colocar zero?
- e se não preencher?
- existe limite máximo?
- pode limpar os dados?
- qual sistema operacional?
- precisa funcionar sem mouse?
- quanto tempo pode demorar para apresentar o resultado?

Tudo isso aparece no exemplo da aula.

🧠 Moral:

"Eu quero um programa que faça X" não significa que os requisitos já estejam definidos.

🟡 PARTE 2 — CONCEITOS TEÓRICOS

Agora vamos começar a falar como uma estudante universitária de Engenharia de Software. 😌📚

1️⃣ TIPOS DE SISTEMAS

A aula relembra três tipos:

🔹 Sistemas embutidos

São sistemas associados a equipamentos/dispositivos.

Nesse caso, é importante compreender:

- a aplicação;
- o problema;
- o ambiente de operação;
- o hardware;
- como aproveitar o equipamento;
- como integrar a solução ao equipamento.

🔹 Sistemas comerciais

São sistemas voltados para necessidades de mercado.

É necessário analisar:

- oportunidades de mercado;
- possíveis usuários;
- necessidades;
- demanda;
- recursos que o sistema oferecerá.

🔹 Sistemas de Informação

Aqui precisamos compreender o **negócio/organização**.

É necessário:

- entender a estrutura da organização;
- compreender sua dinâmica;

- criar entendimento comum entre cliente, usuários e desenvolvedores;
- entender como o novo sistema facilitará os processos.

⚠️ Ideia importante:

O modo de identificar requisitos pode depender do tipo de sistema.

2️⃣ PROCESSOS EMPRESARIAIS

O PDF define processo como:

um conjunto de atividades inter-relacionadas que transforma entradas em saídas.

Pense:

Entrada → atividades → saída

Exemplo:

Pedido do aluno → matrícula → aluno matriculado

Tipos de processos empresariais

A aula apresenta:

🟢 Processos primários

Relacionados aos clientes.

🔵 Processos de suporte

Dão apoio aos processos da organização.

🟣 Processos gerenciais

Relacionados à gestão da organização.

3️⃣ MAPEAMENTO DE PROCESSOS

Antes de desenvolver um sistema, podemos precisar entender como o processo funciona atualmente.

O PDF pergunta, por exemplo:

- Por que precisa ser feito?
- Para quem?
- O que é feito?
- Quais são as entradas?

- Quais regras são seguidas?
- Quais recursos são necessários?
- Quem realiza a atividade?
- É necessária capacitação?
- Existe outra forma de obter o mesmo resultado?
- O processo pode ser otimizado?
- Qual é o custo?

🧠 Em outras palavras:

Antes de criar o sistema:

precisamos entender o problema e o processo que o sistema deverá apoiar.

4️⃣ REQUISITO

Agora chegamos ao coração da aula.

📌 Definição

Requisito é uma capacidade que:

- o usuário precisa para resolver um problema/atingir um objetivo; ou
- deve existir no sistema para satisfazer uma condição formal.

5️⃣ O REQUISITO NÃO DIZ "COMO"

⚠️ **Essa é uma das partes MAIS importantes da aula.**

O requisito deve dizer:

O QUE o sistema deve fazer.

Não:

COMO o sistema será implementado.

O PDF é explícito:

- requisito → **o que o sistema deve fazer**;
- projeto → **como o sistema deve fazer**.

Exemplo:

❌ "O sistema deve usar banco de dados MySQL."

Isso está falando de **como implementar**.

Já:

✅ "O sistema deve permitir cadastrar funcionários."

Isso descreve **o que o sistema deve fazer**.

6️⃣ NECESSIDADE → CARACTERÍSTICA → REQUISITO

Essa parte pode parecer complicada, mas é muito lógica.

Imagine:

👩 Cliente:

"Tenho os dados dos funcionários em papel e quero acessá-los facilmente em um único lugar."

Isso é uma:

💭 NECESSIDADE

O analista transforma essa necessidade em uma:

⭐ CARACTERÍSTICA

"O sistema deverá possuir um cadastro único de funcionários."

E finalmente temos um requisito:

📋 REQUISITO FUNCIONAL

"O sistema deve permitir inclusão, alteração e exclusão dos dados dos funcionários."

Esse exemplo está diretamente no PDF.

7️⃣ REQUISITOS FUNCIONAIS

Agora decore:

🟢 FUNCIONAL = O QUE O SISTEMA FAZ

É uma ação ou funcionalidade que o sistema deverá realizar.

Exemplos:

- cadastrar funcionário;
- alterar funcionário;

- excluir funcionário;
- concluir uma venda;
- emitir nota fiscal;
- atualizar estoque.

O PDF usa exemplos como esses.

🧠 Macete:

Se a frase puder ser entendida como:

"O sistema deve FAZER..."

provavelmente estamos diante de um requisito funcional.

8️⃣ REQUISITOS NÃO FUNCIONAIS

Agora:

🔵 NÃO FUNCIONAL = COMO / RESTRIÇÃO / QUALIDADE

O requisito não funcional estabelece restrições sobre os serviços ou funções oferecidos pelo sistema. O PDF também destaca que devem ser **mensuráveis**, ou seja, deve ser possível verificar se foram atendidos.

Exemplos:

"O sistema deve responder em até 0,5 segundo."

Isso é mensurável.

Não basta dizer:

❌ "O sistema deve ser rápido."

Melhor:

✅ "O sistema deve responder em até 0,5 segundo."

Esse exemplo aparece na aula.

🧠 FUNCIONAL × NÃO FUNCIONAL

Funcional	Não funcional
O que o sistema faz	Restrição/qualidade
Funcionalidade	Característica/condição
"Cadastrar funcionário"	"Responder em até 0,5 s"
"Emitir nota fiscal"	"Somente usuários autenticados"
"Atualizar estoque"	"Utilizar determinado padrão"

9️⃣ CATEGORIAS DOS REQUISITOS

A aula também apresenta três categorias:

🔴 Obrigatório

Deve ser atendido.

Sem ele, o sistema não atende adequadamente ao objetivo.

🟡 Desejável

É altamente desejável, mas **não é obrigatório**.

🟢 Possível

É possível implementar, mas poderia ser eliminado.

🧠 Decore:

Obrigatório = precisa

Desejável = seria bom

Possível = pode ter

🔟 REQUISITOS NÃO FUNCIONAIS — SUBTIPOS

O PDF divide os requisitos não funcionais em:

💻 Requisitos de Produto

Relacionados às características do próprio sistema:

- eficiência;
- desempenho;
- espaço;
- facilidade de uso;
- confiabilidade;
- portabilidade.

🏢 Requisitos Organizacionais

Relacionados à organização:

- entrega;
- implementação;
- padrões.

⚖️ Requisitos Externos

Relacionados a fatores externos:

- interoperabilidade;
- ética;
- aspectos legais;
- privacidade;
- segurança.

1️⃣1️⃣ PROTOTIPAÇÃO

A prototipação aparece novamente nesta aula, agora relacionada aos requisitos.

Ela é útil quando:

- o cliente não tem certeza do que precisa;
- sabe o que quer, mas não consegue explicar;
- o desenvolvedor precisa validar se entendeu corretamente o problema.

Existem dois tipos apresentados:

🗑️ Protótipo descartável

É usado para **explorar/entender**.

Depois, não necessariamente será utilizado no sistema final.

🌱 Protótipo evolutivo

Serve como **base para o desenvolvimento do sistema**.

🔴 PARTE 3 — ENGENHARIA DE REQUISITOS

Agora chegamos ao conceito mais importante da aula.

O que é Engenharia de Requisitos?

É o conjunto de atividades relacionadas à:

produção + gerenciamento dos requisitos.

E ela possui duas grandes partes:

📝 Produção de requisitos

- levantamento;
- registro;
- verificação;
- validação.

🗂️ Gerência de requisitos

- controle de mudanças;
- gerência de configuração;
- rastreabilidade;
- gerência da qualidade.

📝 1. LEVANTAMENTO

É descobrir os requisitos.

Precisamos conversar com:

- clientes;
- usuários finais;
- pessoas envolvidas.

E descobrir:

- qual problema deve ser resolvido;
- quais serviços são necessários;
- qual desempenho é necessário;
- quais são as restrições;
- etc.

Técnicas de levantamento

O PDF apresenta várias:

- entrevistas;
- questionários;
- workshops;
- leitura de documentos;
- brainstorming;
- storyboards;
- job shadowing;
- prototipação;
- etnografia.

🧠 Não precisa decorar todas de primeira.

Mas saiba que existem **diferentes técnicas para descobrir as necessidades do cliente/usuário**.

📄 2. REGISTRO

Depois de descobrir os requisitos:

Precisamos documentá-los.

Por quê?

Porque eles servirão de base para o restante do desenvolvimento.

O registro também facilita:

- controle de alterações;
- criação de versões;
- comunicação entre os stakeholders.

🔎 3. VERIFICAÇÃO

A pergunta aqui é:

"Escrevemos os requisitos corretamente?"

Precisamos verificar se eles estão:

- completos;
- sem ambiguidades;
- sem inconsistências;
- sem omissões.

A aula também cita **inspeções**, que são revisões estruturadas para detectar defeitos.

🤝 4. VALIDAÇÃO

Agora a pergunta muda:

"Esses são realmente os requisitos que o cliente precisa?"

A validação é a atividade em que se obtém o **aceite do cliente** sobre os requisitos especificados.

🧠 Diferença para decorar:

VERIFICAÇÃO = escrevemos certo?

VALIDAÇÃO = escrevemos o que realmente precisava?

Essa diferença é uma excelente candidata a questão de prova. ⭐

🔄 5. GERÊNCIA DE REQUISITOS

Agora vem uma ideia muito importante:

Requisitos mudam!

O PDF chama atenção para o fato de que requisitos são, por natureza, **voláteis**.

Eles podem mudar por:

- mudanças externas no ambiente;
- novas percepções do cliente;
- erros no processo de requisitos.

Imagine:

Você está desenvolvendo um sistema para uma empresa.

No meio do projeto surge uma nova legislação.

O sistema precisa mudar.

Por isso precisamos controlar essas mudanças.

🔄 6. CONTROLE DE MUDANÇAS

Não significa:

"O cliente pediu, então coloca."

É necessário um processo formal.

A aula apresenta:

solicitação → avaliação → aprovação → implementação

E todas as mudanças devem ser rastreadas.

🧠 Por quê?

Para não perder o controle de:

- escopo;
- prazo;
- custo.

🗃️ 7. GERÊNCIA DE CONFIGURAÇÃO

Aqui a preocupação é com as **versões dos artefatos**.

Imagine:

👩 Ana está trabalhando na versão 3.

👨 João está trabalhando na versão 2.

👩 Ana altera alguma coisa.

👨 João continua trabalhando na versão antiga.

😢

Problema!

A gerência de configuração busca manter sob controle as diferentes versões e garantir que a equipe compartilhe a versão correta dos artefatos.

🧠 Palavra-chave:

VERSÕES

🔗 8. RASTREABILIDADE

Esse nome assusta, mas a ideia é simples.

É conseguir **ligar um requisito àquilo que deu origem a ele e ao que foi produzido a partir dele**.

O PDF mostra relações entre:

Necessidades do cliente/documentos

⬇️

Requisitos

⬇️

Projeto/implementação

⬇️

Casos de teste

Essas ligações podem existir nos dois sentidos.

🔙 Pré-rastreabilidade

Liga o requisito àquilo que veio **antes** dele.

Exemplo:

necessidade do cliente → requisito

🔜 Pós-rastreabilidade

Liga o requisito àquilo que veio **depois**.

Exemplo:

requisito → componente/caso de teste

🧠 Macete:

PRÉ = antes

PÓS = depois

⭐ 9. QUALIDADE DOS REQUISITOS

A Engenharia de Requisitos também precisa garantir que temos uma boa base de requisitos.

O PDF destaca que devemos verificar se eles:

- estão completos;
- são claros;
- podem ser implementados;
- são consistentes;
- são verificáveis;
- possuem problemas potenciais identificados;
- foram negociados com os grupos envolvidos.

🎓 PARTE 4 — COMO ISSO PODE APARECER NA PROVA

Agora vamos transformar tudo em respostas de nível universitário.

❓ QUESTÃO 1 — O que é um requisito?

Uma resposta completa seria:

Resposta de prova — Requisito

Um requisito representa uma capacidade necessária ao usuário para resolver um problema ou atingir determinado objetivo, ou uma capacidade que deve existir no sistema para atender a contratos, padrões, especificações ou outras condições formalmente estabelecidas. O requisito deve expressar o que o sistema deve fazer, sem antecipar a forma de implementação da solução.

❓ QUESTÃO 2 — Qual a diferença entre requisito e projeto?

Resposta:

Resposta de prova — Requisito x Projeto

O requisito define o que o sistema deve fazer, concentrando-se nas necessidades do cliente e no problema a ser solucionado. O projeto, por sua vez, define como o sistema realizará essas funções, tratando da solução e de sua implementação. Portanto, discutir a solução antes de compreender claramente o problema e seus requisitos é prematuro.

❓ QUESTÃO 3 — Diferencie requisito funcional e não funcional.

Resposta de prova — Requisitos Funcionais e Não Funcionais

Requisitos funcionais descrevem as funcionalidades ou ações que o sistema deve realizar, ou seja, especificam o que o sistema deve fazer. Requisitos não funcionais estabelecem restrições, características ou condições relacionadas aos serviços oferecidos pelo sistema, indicando aspectos como desempenho, segurança, confiabilidade, portabilidade, privacidade ou padrões. Os requisitos não funcionais devem ser mensuráveis e verificáveis.

❓ QUESTÃO 4 — Diferencie verificação e validação.

🧠 Pense:

Verificação → requisito escrito corretamente.

Validação → requisito correto para o cliente.

Resposta:

Resposta de prova — Verificação x Validação

A verificação dos requisitos busca assegurar que eles foram definidos corretamente, sem ambiguidades, inconsistências ou omissões. A validação, por sua vez, busca obter o aceite do cliente sobre os requisitos especificados, verificando se aquilo que foi definido corresponde efetivamente às necessidades do cliente e do sistema.

❓ QUESTÃO 5 — Por que os requisitos precisam ser gerenciados?

Porque eles são **voláteis**.

Eles podem mudar devido a:

- ambiente externo;
- novas percepções do cliente;
- erros na identificação dos requisitos.

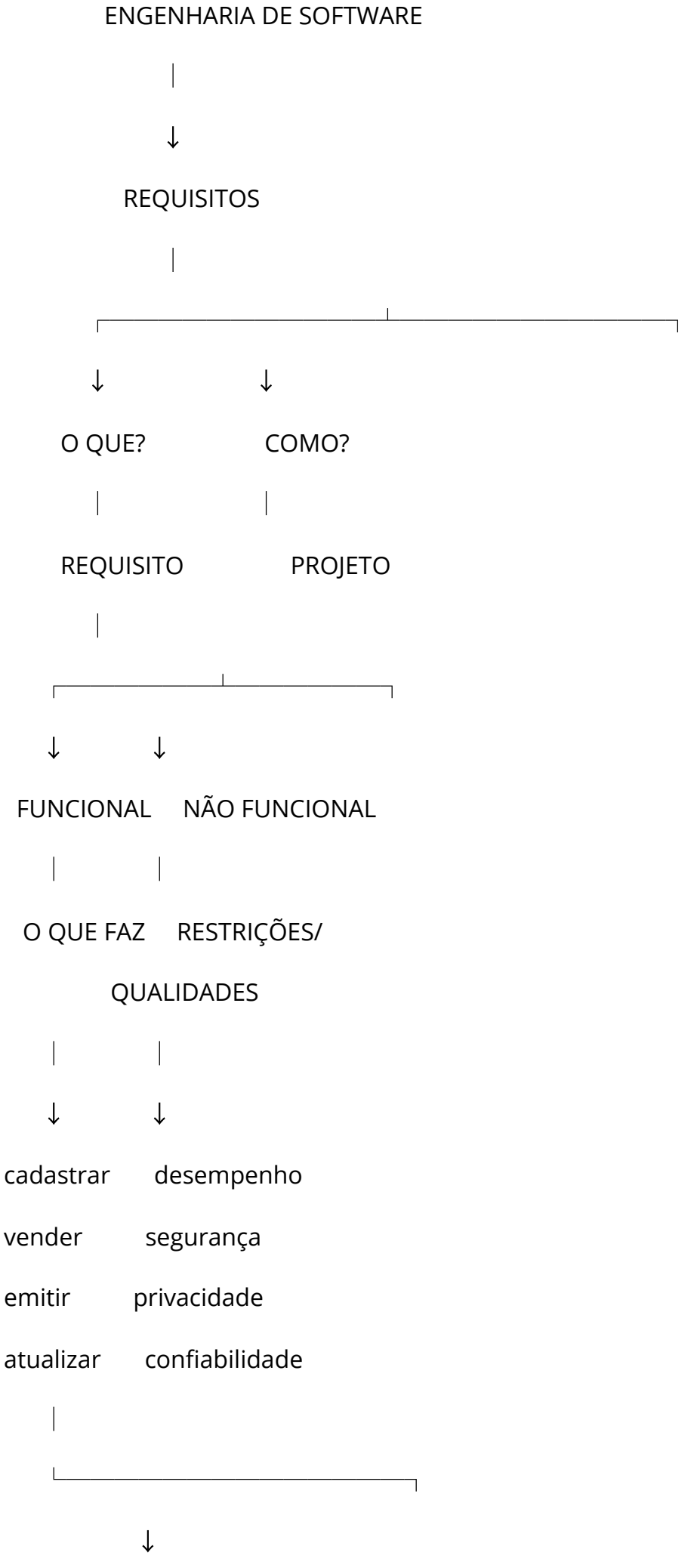
Por isso, as mudanças precisam ser controladas para evitar perda de controle sobre **escopo, prazo e custo**.

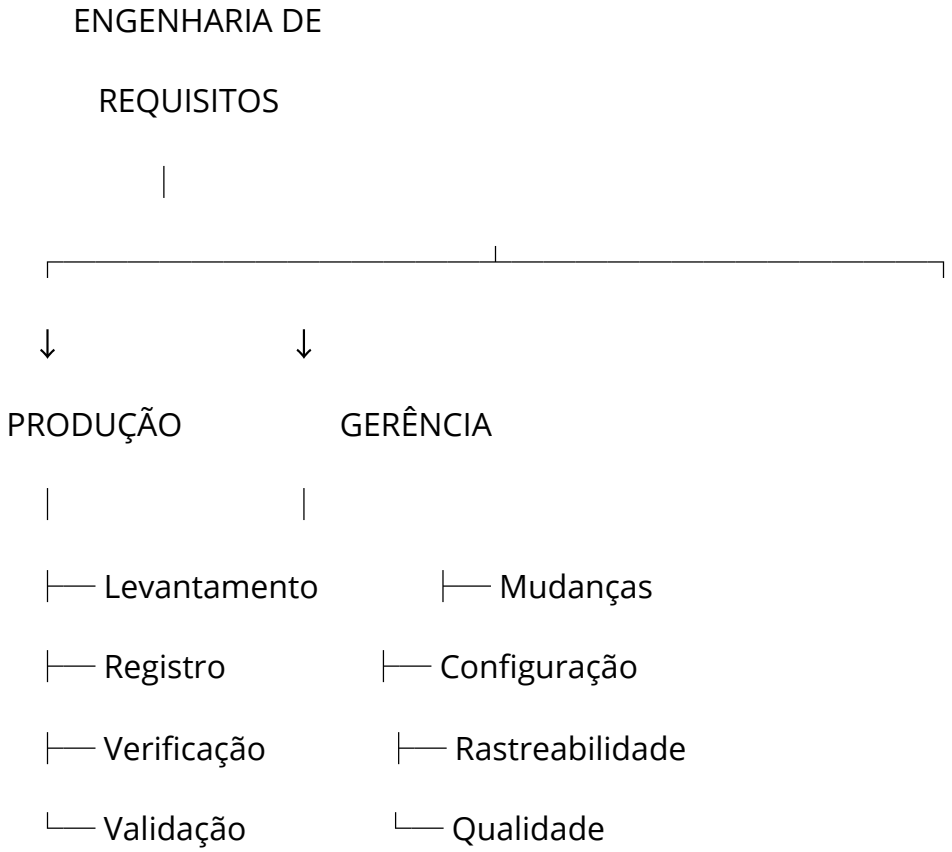
❓ QUESTÃO 6 — O que é rastreabilidade?

Resposta de prova — Rastreabilidade

Rastreabilidade é a capacidade de estabelecer ligações entre as fontes dos requisitos, os próprios requisitos e outros artefatos do desenvolvimento, como componentes, artefatos de projeto, implementação e casos de teste. A rastreabilidade pode ocorrer tanto em direção às origens do requisito quanto em direção aos artefatos derivados dele.

🧠 MAPA MENTAL DA AULA 3


ENGENHARIA DE SOFTWARE
REQUISITOS
O QUE?
COMO?
REQUISITO
PROJETO
FUNCIONAL
NÃO FUNCIONAL
O QUE FAZ
RESTRIÇÕES/
QUALIDADES
cadastrar
vender
emitir
atualizar
desempenho
segurança
privacidade
confiabilidade



🚨 AS 10 COISAS QUE EU DECORARIA PARA A PROVA

Se você estiver com pouco tempo, **priorize estas**:

1. ⭐ Requisito

O que o sistema deve fazer.

2. ⭐ Projeto

Como o sistema fará.

3. ⭐ Funcional

O que o sistema faz.

4. ⭐ Não funcional

Restrição/característica/qualidade e deve ser mensurável.

5. ⭐ Obrigatório

Precisa ter.

6. ⭐ Desejável

Seria muito bom ter, mas não é obrigatório.

7. ⭐ Possível

Pode ser implementado, mas pode ser eliminado.

8. ⭐ Verificação

Está escrito corretamente?

9. ⭐ Validação

É realmente o que o cliente precisa?

10. ⭐ Rastreabilidade

Liga necessidade → requisito → projeto/implementação/testes.

🎯 E UMA PEGADINHA MUITO IMPORTANTE

Veja estas frases:

A) "O sistema deve permitir cadastrar alunos."

➡️ 🟢 **FUNCIONAL**

B) "O sistema deve responder em até 0,5 segundo."

➡️ 🔵 **NÃO FUNCIONAL**

C) "O sistema deve utilizar C++."

➡️ 🔵 **NÃO FUNCIONAL/restrição**

D) "O sistema deve emitir nota fiscal eletrônica."

➡️ 🟢 **FUNCIONAL**

E) "Os dados só podem ser acessados por usuários autenticados."

➡️ 🔵 **NÃO FUNCIONAL**, relacionado à segurança.

Esses exemplos estão entre os exercícios apresentados no próprio PDF.

💡 A IDEIA CENTRAL DA AULA 3

Se eu tivesse que explicar **a aula inteira em uma única frase**, seria:

Antes de construir um software, precisamos entender profundamente o problema e descobrir, documentar, verificar, validar e gerenciar aquilo que o cliente realmente precisa.

E existe uma sequência mental que pode te salvar na prova:

NECESSIDADE

⬇️

"Preciso resolver um problema."

REQUISITO

⬇️

"O sistema precisa fazer/possuir isso."

PROJETO

⬇️

"Vamos decidir como fazer."

IMPLEMENTAÇÃO

⬇️

"Vamos construir."

TESTES

⬇️

"Vamos verificar."

E durante tudo isso:

GERÊNCIA DE REQUISITOS

⬇️

"Vamos controlar mudanças, versões, rastreabilidade e qualidade."